

Лабораторная работа № 2

Тема: Полный CRUD Product API с PostgreSQL

Цель работы: Закрепить базовые команды SQL и полностью перевести CRUD-операции Product API с JavaScript-массива на PostgreSQL.

После выполнения все операции:

- Create
- Read
- Update
- Delete

должны работать с таблицей products

1. Проверьте исходную таблицу

Используем таблицу из предыдущей лабораторной:

```
CREATE TABLE products (  
  id INTEGER  
    GENERATED ALWAYS AS IDENTITY  
    PRIMARY KEY,  
  
  name VARCHAR(150)  
    NOT NULL,  
  
  price INTEGER  
    NOT NULL  
    CHECK (price > 0),  
  
  category VARCHAR(100)  
    NOT NULL,  
  
  in_stock BOOLEAN  
    NOT NULL  
    DEFAULT true  
);
```

2. Проверьте содержимое таблицы

```
SELECT  
  id,  
  name,  
  price,  
  category,  
  in_stock  
FROM products
```

ORDER BY id;

3. Выполните **SELECT** с условием

Получите товары дороже 10000:

```
SELECT
  id,
  name,
  price
FROM products
WHERE price > 10000;
```

4. Добавьте условие наличия

```
SELECT
  id,
  name,
  price
FROM products
WHERE price > 10000
  AND in_stock = true;
```

5. Отсортируйте результат

```
SELECT
  id,
  name,
  price
FROM products
WHERE in_stock = true
ORDER BY price ASC;
```

6. Выполните **INSERT** вручную

```
INSERT INTO products
(
  name,
  price,
  category,
  in_stock
)
VALUES
(
  'SSD 2TB',
  65000,
  'Components',
  true
);
```

Проверьте:
SELECT *
FROM products
ORDER BY id;

7. Используйте RETURNING

Добавьте ещё один товар:

```
INSERT INTO products
```

```
(  
    name,  
    price,  
    category,  
    in_stock
```

```
)
```

```
VALUES
```

```
(  
    'Web Camera',  
    22000,  
    'Accessories',  
    true
```

```
)
```

```
RETURNING
```

```
id,  
name,  
price,  
category,  
in_stock;
```

RETURNING в PostgreSQL позволяет INSERT, UPDATE и DELETE сразу вернуть данные изменённых строк, поэтому дополнительный SELECT часто не требуется.

8. Выполните UPDATE вручную

Выберите id созданного товара и измените его цену:

```
UPDATE products
```

```
SET price = 20000
```

```
WHERE id = 5
```

```
RETURNING *;
```

Используйте фактический id вашего товара.

9. Проверьте опасность UPDATE без WHERE

Посмотрите на команду:

```
UPDATE products
```

```
SET price = 20000;
```

Не выполняйте её.

Она изменит цену всех строк таблицы.

Правильный вариант:

```
UPDATE products
```

```
SET price = 20000
```

```
WHERE id = 5;
```

10. Выполните DELETE вручную

```
DELETE FROM products
```

```
WHERE id = 5
```

```
RETURNING *;
```

Убедитесь, что удалена только нужная строка.

11. Откройте productService.js

После предыдущей лабораторной в Service уже должны использоваться:

```
getAllProducts()
```

```
findProductById()
```

через PostgreSQL.

Теперь необходимо реализовать:

```
addProduct()
```

```
updateProductById()
```

```
deleteProductById()
```

12. Проверьте импорт Pool

В начале:

```
import {
```

```
  pool
```

```
} from "../config/db.js";
```

13. Реализуйте получение всех товаров

```
export async function getAllProducts() {
```

```
  const result =
```

```
    await pool.query(
```

```
      `SELECT
```

```
        id,
```

```
        name,
```

```
        price,
```

```
        category,
```

```
        in_stock AS "inStock"
```

```
      FROM products
```

```
      ORDER BY id`
```

```
    );
```

```
    return result.rows;
}
```

14. Реализуйте получение одного товара

```
export async function findProductById(
  id
){
  const result =
    await pool.query(
      `SELECT
        id,
        name,
        price,
        category,
        in_stock AS "inStock"
      FROM products
      WHERE id = $1`,
      [id]
    );

  return result.rows[0];
}
```

Параметризованные запросы node-postgres передают SQL и значения отдельно; документация рекомендует этот подход вместо конкатенации пользовательских значений со строкой SQL.

15. Реализуйте CREATE

Добавьте в productService.js:

```
export async function addProduct(
  data
){
  const result =
    await pool.query(
      `INSERT INTO products
      (
        name,
        price,
        category,
        in_stock
      )
      VALUES
      ($1, $2, $3, $4)
      RETURNING
      id,
```

```

        name,
        price,
        category,
        in_stock AS "inStock",
    [
        data.name,
        data.price,
        data.category,
        data.inStock
    ]
);

return result.rows[0];
}

```

16. Проверьте соответствие параметров

В SQL:

\$1 → name

\$2 → price

\$3 → category

\$4 → inStock

В массиве:

```

[
    data.name,
    data.price,
    data.category,
    data.inStock
]

```

Порядок должен совпадать.

17. Проверьте Controller создания

```

export async function createProduct(
    req,
    res,
    next
) {
    try {
        const product =
            await addProduct(req.body);

        res
            .status(201)
            .json(product);
    } catch (error) {

```

```
    next(error);  
  }  
}
```

18. Выполните POST

POST /api/products

Тело:

```
{  
  "name": "Gaming Mouse",  
  "price": 18000,  
  "category": "Accessories",  
  "inStock": true  
}
```

Ожидается:

201 Created

Пример ответа:

```
{  
  "id": 6,  
  "name": "Gaming Mouse",  
  "price": 18000,  
  "category": "Accessories",  
  "inStock": true  
}
```

Фактический id зависит от состояния вашей таблицы.

19. Проверьте PostgreSQL

Выполните:

SELECT *

FROM products

ORDER BY id;

Товар, созданный через REST API, должен находиться в таблице.

20. Реализуйте UPDATE

Для существующего PATCH API используйте:

export async function updateProductById(
 id,
 changes
) {

```
  const result =  
    await pool.query(  
      `UPDATE products  
      SET  
      name =  
      COALESCE($1, name),
```

```

    price =
        COALESCE($2, price),

    category =
        COALESCE($3, category),

    in_stock =
        COALESCE($4, in_stock)

WHERE id = $5

RETURNING
    id,
    name,
    price,
    category,
    in_stock AS "inStock",
[
    changes.name ?? null,
    changes.price ?? null,
    changes.category ?? null,
    changes.inStock ?? null,
    id
]
);

```

```

    return result.rows[0];
}

```

В данной учебной таблице поля name, price, category и in_stock имеют ограничения NOT NULL, поэтому null используется здесь только как технический признак: «это поле не было передано в PATCH».

21. Разберите COALESCE

Например:

```
price = COALESCE($2, price)
```

Если \$2 содержит:

42000

результат:

```
price = 42000
```

Если \$2 содержит:

NULL

используется текущее значение:

```
price = price
```


22. Проверьте Controller UPDATE

```
export async function updateProduct(
  req,
  res,
  next
) {
  try {
    const id =
      Number(req.params.id);

    const product =
      await updateProductById(
        id,
        req.body
      );

    if (!product) {
      return next(
        new AppError(
          "Product not found",
          404
        )
      );
    }

    res.json(product);
  } catch (error) {
    next(error);
  }
}
```

23. Выполните PATCH

PATCH /api/products/6

Используйте фактический id.

Тело:

```
{
  "price": 16500
}
```

Ожидаемый результат: изменяется только цена.

Например:

```
{
  "id": 6,
  "name": "Gaming Mouse",
```

```
"price": 16500,  
"category": "Accessories",  
"inStock": true  
}
```

24. Проверьте изменения непосредственно в PostgreSQL

```
SELECT *
```

```
FROM products
```

```
WHERE id = 6;
```

Цена должна соответствовать PATCH-запросу.

25. Реализуйте DELETE

```
export async function deleteProductById(  
  id  
){
```

```
  const result =
```

```
    await pool.query(  
      `DELETE FROM products  
        WHERE id = $1  
        RETURNING  
          id,  
          name,  
          price,  
          category,  
          in_stock AS "inStock",  
          [id]  
    `);
```

```
  return result.rows[0];  
}
```

26. Проверьте Controller DELETE

```
export async function deleteProduct(  
  req,  
  res,  
  next  
){
```

```
  try {
```

```
    const id =
```

```
      Number(req.params.id);
```

```
    const product =
```

```
      await deleteProductById(id);
```

```

    if (!product) {
      return next(
        new AppError(
          "Product not found",
          404
        )
      );
    }

    res.json(product);
  } catch (error) {
    next(error);
  }
}

```

27. Выполните DELETE

DELETE /api/products/6

Ожидается JSON удалённого товара.

28. Проверьте отсутствие строки

GET /api/products/6

Ожидается:

404 Not Found

Затем выполните непосредственно в PostgreSQL:

SELECT *

FROM products

WHERE id = 6;

Результат должен содержать:

0 строк

29. Выполните полный CRUD-цикл

Create

POST /api/products

↓

INSERT

Read

GET /api/products/:id

↓

SELECT

Update

PATCH /api/products/:id

↓

UPDATE

Delete

DELETE /api/products/:id

↓

DELETE

30. Проверьте постоянное хранение

Создайте новый товар через API.

Остановите Node.js:

Ctrl + C

Запустите Express снова.

Выполните:

GET /api/products

Товар должен сохраниться, потому что теперь он находится в PostgreSQL.

Итоговый productService.js

```
import {
  pool
} from "../config/db.js";

export async function getAllProducts() {
  const result =
    await pool.query(
      `SELECT
        id,
        name,
        price,
        category,
        in_stock AS "inStock"
      FROM products
      ORDER BY id`
    );

  return result.rows;
}

export async function findProductById(
  id
) {
  const result =
    await pool.query(
      `SELECT
        id,
        name,
        price,
```

```

        category,
        in_stock AS "inStock"
    FROM products
    WHERE id = $1`,
    [id]
);

return result.rows[0];
}

export async function addProduct(
    data
) {
    const result =
        await pool.query(
            `INSERT INTO products
            (
                name,
                price,
                category,
                in_stock
            )
            VALUES
            ($1, $2, $3, $4)
            RETURNING
            id,
            name,
            price,
            category,
            in_stock AS "inStock"`,
            [
                data.name,
                data.price,
                data.category,
                data.inStock
            ]
        );

    return result.rows[0];
}

export async function updateProductById(
    id,
    changes

```

```

) {
  const result =
    await pool.query(
      `UPDATE products
      SET
        name =
          COALESCE($1, name),
        price =
          COALESCE($2, price),
        category =
          COALESCE($3, category),
        in_stock =
          COALESCE($4, in_stock)
      WHERE id = $5
      RETURNING
        id,
        name,
        price,
        category,
        in_stock AS "inStock"`,
      [
        changes.name ?? null,
        changes.price ?? null,
        changes.category ?? null,
        changes.inStock ?? null,
        id
      ]
    );

  return result.rows[0];
}

export async function deleteProductById(
  id
) {
  const result =
    await pool.query(
      `DELETE FROM products
      WHERE id = $1
      RETURNING
        id,
        name,
        price,
        category,

```

```
        in_stock AS "inStock",  
        [id]  
    );  
  
    return result.rows[0];  
}
```

Для одиночных SQL-команд `pool.query()` сам получает свободный `client` из `Pool` и возвращает его обратно после выполнения запроса; отдельный `client.release()` здесь не требуется.

Требования к результату

Приложение должно:

- получать товары через `SELECT`;
- получать один товар через `SELECT ... WHERE`;
- создавать товар через `INSERT`;
- использовать `RETURNING` после `INSERT`;
- изменять товар через `UPDATE`;
- поддерживать частичный `PATCH`;
- удалять товар через `DELETE`;
- использовать `WHERE` для `UPDATE` и `DELETE`;
- использовать параметризованные запросы;
- не содержать массива `products[]`;
- хранить SQL только в `Service`;
- возвращать 404 для отсутствующего `id`;
- сохранять данные после перезапуска `Node.js`